

全国青少年信息学奥林匹克模拟赛

NOIP 2025

时间：2025 年 11 月 27 日 07:30 ~ 12:00

题目名称	原根	更优的背包	简单题	拍照
题目类型	传统型	传统型	传统型	传统型
目录	root	knapsack	easy	photo
可执行文件名	root	knapsack	easy	photo
输入文件名	root.in	knapsack.in	easy.in	photo.in
输出文件名	root.out	knapsack.out	easy.out	photo.out
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	4.0 秒
内存限制	1024 MiB	1024 MiB	1024 MiB	1024 MiB
子任务数目	10	7	5	7
子任务是否等分	是	否	否	否

提交源程序文件名

对于 C++ 语言	root.cpp	knapsack.cpp	easy.cpp	photo.cpp
-----------	----------	--------------	----------	-----------

编译选项

对于 C++ 语言	-O2 -std=c++14
-----------	----------------

注意事项（请仔细阅读）

1. 文件名（包括程序名和输入输出文件名）必须使用英文小写。
2. C++ 中函数 `main()` 的返回值类型必须是 `int`，值必须为 0。
3. 对于因未遵守以上规则对成绩造成的影响，相关申诉不予受理。
4. 若无特殊说明，输入文件中同一行内的多个整数、浮点数、字符串等均使用一个空格进行分隔。
5. 若无特殊说明，结果比较方式为忽略行末空格、文末回车后的全文比较。
6. 程序可使用的栈空间大小与该题内存空间限制一致。
7. 在终端下可使用命令 `ulimit -s unlimited` 将栈空间限制放大，但你使用的栈空间大小不应超过题目限制。

原根 (root)

【题目描述】

给定 n, m , 求

$$\sum_{i=0}^m [i \oplus (n-1) \equiv 1 \pmod{n}]$$

【输入格式】

从文件 `root.in` 中读入数据。

本题包含多组测试数据。

第一行包含一个整数 T , 表示测试数据组数。

对于每组测试数据:

第一行包含两个整数 n, m , 含义如题面所示。

【输出格式】

输出到文件 `root.out` 中。

对于每组测试数据, 输出一行一个整数表示答案。

【样例 1 输入】

```
1 3
2 2 0
3 7 11
4 1145141 998244353
```

【样例 1 输出】

```
1 1
2 2
3 872
```

【样例 2】

见选手目录下的 `root/root2.in` 和 `root/root2.ans`。

该样例满足 $T = 10^5$, $\lfloor \frac{m}{n} \rfloor \leq 10^5$ 。

【数据范围】

对于 30% 的数据满足, $\sum m \leq 10^5$ 。

对于另外 30% 的数据满足, $\sum \lfloor \frac{m}{n} \rfloor \leq 10^5$ 。

对于 100% 的数据满足, $1 \leq T \leq 10^5$, $2 \leq n \leq 10^{18}$, $0 \leq m \leq 10^{18}$ 。

更优的背包 (knapsack)

【题目描述】

有 n 个物品，其中第 i 个物品的价值为 v_i ，质量为 m_i 。你可以选择若干物品（每个物品只能选择一次），并求出质量之和不超过 M 的情况下价值之和的最大值。

保证对于任意 i, j ，要么 $m_i \mid m_j$ ，要么 $m_j \mid m_i$ 。

【输入格式】

从文件 *knapsack.in* 中读入数据。

第一行包含两个整数 n 和 M ，含义如题面所示。

接下来的 n 行，每行包含两个整数 v_i 和 m_i ，分别表示第 i 个物品的价值和质量。

【输出格式】

输出到文件 *knapsack.out* 中。

输出一行一个整数表示答案。

【样例 1 输入】

```
1 6 10
2 1 1
3 5 2
4 200 6
5 9 2
6 6 2
7 100 1
```

【样例 1 输出】

```
1 310
```

【样例 2】

见选手目录下 *knapsack/knapsack2.in* 与 *knapsack/knapsack2.ans*。

该样例满足 Subtask 2 的限制。

【样例 3】

见选手目录下 *knapsack/knapsack3.in* 与 *knapsack/knapsack3.ans*。
该样例满足 Subtask 3 的限制。

【样例 4】

见选手目录下 *knapsack/knapsack4.in* 与 *knapsack/knapsack4.ans*。
该样例满足 Subtask 5 的限制。

【样例 5】

见选手目录下 *knapsack/knapsack5.in* 与 *knapsack/knapsack5.ans*。
该样例满足 Subtask 6 的限制。

【样例 6】

见选手目录下 *knapsack/knapsack6.in* 与 *knapsack/knapsack6.ans*。
该样例满足 Subtask 7 的限制。

【数据范围】

本题开启捆绑测试。

- Subtask 1 (8 pts): 保证 $n = 2$, $M \leq 10^4$ 。
- Subtask 2 (8 pts): 保证 $n \leq 20$, $M \leq 10^4$ 。
- Subtask 3 (16 pts): 保证 $n \leq 1000$, $M \leq 10^4$ 。
- Subtask 4 (24 pts): 保证 $n \leq 1000$, $M \leq 10^8$ 。
- Subtask 5 (8 pts): 保证 $M \leq 10^8$, 所有 m_i 相等。
- Subtask 6 (12 pts): 保证 $M \leq 10^8$, 最多两种不同的 m_i 。
- Subtask 7 (24 pts): 无特殊限制。

对于所有数据, 保证 $1 \leq n \leq 5 \times 10^5$, $1 \leq M, v_i, m_i \leq 10^{12}$ 。

简单题 (easy)

【题目描述】

给定长度为 N ，值域为 $[0, K)$ 的序列 A 。你可以执行以下操作任意次（可能为 0 次）：

- 选择 i, x, y 满足 $1 \leq i < N$, $0 \leq x, y < K$;
- 设 $z = (x \cdot A_i + y \cdot A_{i+1}) \bmod K$ ，将 A_i 和 A_{i+1} 替换成 z （即将 A_i 和 A_{i+1} 删除，并在 A_{i-1} 和 A_{i+2} 之间插入 z ）。

你需要求出有多少序列能通过上述操作生成。答案对 998244353 取模。

【输入格式】

从文件 `easy.in` 中读入数据。

本题包含多组测试数据。

第一行包含一个整数 T ，表示测试数据组数。

对于每组测试数据：

第一行包含两个整数 N, K ，表示序列的长度和值域。

第二行包含 N 个整数 $A_1, A_2, \dots, A_N$ ，表示序列 A 。

【输出格式】

输出到文件 `easy.out` 中。

对于每组测试数据，输出一行表示答案对 998244353 取模后的结果。

【样例 1 输入】

```

1 5
2 2 2
3 0 1
4 2 3
5 2 2
6 4 5
7 0 0 0 0
8 3 6
9 2 4 3
10 8 1000
11 1 1 1 1 1 1 1 1
    
```

【样例 1 输出】

```
1 3
2 4
3 4
4 15
5 772713765
```

【样例 1 解释】

对于第一组测试数据，序列 $[0, 1]$ ， $[0]$ 和 $[1]$ 可以被生成。

【样例 2】

见选手目录下的 *easy/easy2.in* 和 *easy/easy2.ans*。
该样例满足 Subtask 1 的限制。

【样例 3】

见选手目录下的 *easy/easy3.in* 和 *easy/easy3.ans*。
该样例满足 Subtask 2 的限制。

【样例 4】

见选手目录下的 *easy/easy4.in* 和 *easy/easy4.ans*。
该样例满足 Subtask 3 的限制。

【样例 5】

见选手目录下的 *easy/easy5.in* 和 *easy/easy5.ans*。
该样例满足 Subtask 4 的限制。

【样例 6】

见选手目录下的 *easy/easy6.in* 和 *easy/easy6.ans*。
该样例满足 Subtask 5 的限制。

【数据范围】

本题开启捆绑测试。

- Subtask 1 (10 pts): 保证 $N = 2$ 。

- Subtask 2 (30 pts): 保证 $\sum N \leq 9$, $K \leq 10^3$ 。
- Subtask 3 (20 pts): 保证 $\gcd(A_i, A_{i+1}) = 1$ 。
- Subtask 4 (10 pts): 保证 $\sum N \leq 10^3$ 。
- Subtask 5 (30 pts): 无特殊限制。

对于全部数据满足, $1 \leq T \leq 10^4$, $2 \leq N$, $\sum N \leq 2 \times 10^5$, $2 \leq K \leq 10^9$, $0 \leq A_i < K$ 。

拍照 (photo)

【题目描述】

小 A 要拍 n 张照片，拍摄一张照片需要花费 t 的时间。

第 i 张照片最早在时刻 a_i **开始拍摄**，最晚在时刻 b_i **前完成拍摄**。也就是说，设开始拍摄第 i 张照片的时间为 c_i ，需要满足 $[c_i, c_i + t) \subseteq [a_i, b_i)$ 。

同一个时刻只能拍摄一张照片，即对于任意 $i \neq j$ ， $[c_i, c_i + t) \cap [c_j, c_j + t) = \emptyset$ 。你需要求出是否能拍摄所有照片，并构造方案。

【输入格式】

从文件 `photo.in` 中读入数据。

第一行包含两个整数 n, t ，分别表示需要拍摄的照片数量和拍摄一张照片的时间。

接下来 n 行，每行两个整数 a_i, b_i ，表示第 i 张照片最早在时刻 a_i 开始拍摄，最晚在时刻 b_i 前完成拍摄。

【输出格式】

输出到文件 `photo.out` 中。

- 如果能拍摄所有照片，在第一行输出 **yes**，并在第二行输出 n 个整数，第 i 个整数表示**开始拍摄**第 i 张照片的时刻，若有多组方案，输出任意一种即可。
- 如果不能拍摄所有照片，在第一行输出 **no**。

为了避免透露构造方法，下发的样例中仅包含第一行的 **yes** 或 **no**，不包含具体的构造方法。但你需要在存在解的情况下输出具体方案，而不能只输出一行一个 **yes**。

【样例 1 输入】

```
1 2 10
2 0 15
3 5 20
```

【样例 1 输出】

```
1 yes
2 0 10
```

【样例 2 输入】

```
1 2 10
2 1 15
3 0 20
```

【样例 2 输出】

```
1 no
```

【样例 3 输入】

```
1 2 10
2 5 30
3 10 20
```

【样例 3 输出】

```
1 yes
2 20 10
```

【样例 4】

见选手目录下的 *photo/photo4.in* 和 *photo/photo4.ans*。
该样例满足 Subtask 1 的限制。

【样例 5】

见选手目录下的 *photo/photo5.in* 和 *photo/photo5.ans*。
该样例满足 Subtask 2 的限制。

【样例 6】

见选手目录下的 *photo/photo6.in* 和 *photo/photo6.ans*。
该样例满足 Subtask 3 的限制。

【样例 7】

见选手目录下的 *photo/photo7.in* 和 *photo/photo7.ans*。
该样例满足 Subtask 4 的限制。

【样例 8】

见选手目录下的 *photo/photo8.in* 和 *photo/photo8.ans*。

该样例满足 Subtask 5 的限制。

【样例 9】

见选手目录下的 *photo/photo9.in* 和 *photo/photo9.ans*。

该样例满足 Subtask 6 的限制。

【数据范围】

本题开启捆绑测试。

- Subtask 1 (10 pts): 保证 $n \leq 10$ 。
- Subtask 2 (10 pts): 保证 $n \leq 20$ 。
- Subtask 3 (20 pts): 保证 $t = 1$ 。
- Subtask 4 (20 pts): 保证 $n \leq 50$ 。
- Subtask 5 (10 pts): 保证 $n \leq 500$ 。
- Subtask 6 (10 pts): 保证 $n \leq 3000$ 。
- Subtask 7 (20 pts): 无特殊限制。

对于所有数据, 保证 $1 \leq n \leq 10^4$, $1 \leq t \leq 10^5$, $0 \leq a_i \leq a_i + t \leq b_i \leq 10^9$ 。

【评分方式】

本题评分方式如下:

- 若错误判掉了能否拍摄所有照片, 得 0% 的分数。
- 若正确判掉了能否拍摄所有照片, 但构造的方案错误, 得 60% 的分数。
- 若正确判掉了能否拍摄所有照片, 且构造的方案正确, 得 100% 的分数。

需要注意的是, 就算只回答能否拍摄所有照片, 也需要构造一个方案, 否则只能得 0% 的分数。